Axcelson O. Macansantos
BSIT-3 / IT342 – G4

April 6, 2026

SOFTWARE DESIGN PATTERNS REPORT

REPORT 1 — Factory Pattern

Original implementation

BookingController.createBooking() created a Booking object by writing many new Booking() and set…() lines directly inside the controller.
AuthService.register() also created a User object the same way.

Problems

- The controller/service was doing too much. It handled both logic and object creation.
- If you add a new required field, you must update every place that creates that object.
- The same object creation code could be repeated in other methods.

Applied pattern:

```
// BookingController.java (BEFORE)
Booking booking = new Booking();
booking.setClientId(UUID.fromString((String) request.get("clientId")));
booking.setTechnicianId(UUID.fromString((String)
request.get("technicianId")));
booking.setServiceType((String) request.get("serviceType"));
booking.setDeviceType((String) request.get("deviceType"));
// ... 13 more lines of setters ...
booking.setStatus("pending");
booking.setCreatedAt(LocalDateTime.now());
Booking saved = bookingRepository.save(booking);
```

Factory Pattern:

```
// BookingFacade.java (AFTER)
Booking booking = BookingFactory.create(request);  // Factory Pattern
Booking saved = bookingRepository.save(booking);
```

Where it was applied

- BookingFactory used in BookingFacade.createBooking()
- UserFactory used in AuthService.register()

Why this pattern fits
The Factory pattern is a good choice when you want one class to handle object creation. Instead of building objects manually in many places, the code asks the factory to create the object.

Improvement

- Controller and service code became shorter.
- Object creation can now be tested separately.
- If a new field is added to Booking, only the factory usually needs to change.

REPORT 2 — Builder Pattern

Original implementation
LoginResponse was built using many set...() lines.
That same code was repeated in different parts of the system.

Problems

- The same response-building code appeared in multiple places.
- Adding a new field meant updating all those places.
- The code was harder to read because the setter lines were scattered.
- A setter could easily be forgotten, which could cause missing data.

Applied pattern:

```java
// AuthService.java (BEFORE)
LoginResponse response = new LoginResponse();
response.setName(user.getName());
response.setEmail(user.getEmail());
response.setRole(user.getRole());
response.setContactNo(user.getContactNo());
response.setVerified(user.getVerified());
response.setMessage("Login successful");
response.setToken(token);
return response;
```

Builder Pattern:

```java
// AuthService.java (AFTER)
return new LoginResponse.Builder()
        .name(user.getName())
        .email(user.getEmail())
        .role(user.getRole())
        .contactNo(user.getContactNo())
        .verified(user.getVerified())
        .message("Login successful")
        .token(token)
        .build();
```

Where it was applied

- LoginResponse.Builder inside LoginResponse
- Used in AuthService.login()
- Used in AuthController.completeOAuthRegistration()

Why this pattern fits
The Builder pattern works well when an object has many fields and must be built in a clean, readable way.

Improvement

- All code now builds LoginResponse in one consistent style.
- The code is easier to read because it clearly shows how the response is being built.
- Adding a new field is easier because the builder handles it in one place.

REPORT 3 — Facade Pattern

Original implementation
BookingController directly used both BookingRepository and UserRepository.
It also handled extra work like getting technician/client names and adding them to the response inside the controller methods.

Problems

- The controller had too many responsibilities.
- It handled HTTP requests and business/data logic at the same time.
- The same enrichment logic could be repeated in other controllers.
- The controller became too long and harder to manage.

Applied pattern:

```
// BookingController.java (BEFORE) — direct repository access + enrichment
in controller
private final BookingRepository bookingRepository;
private final UserRepository userRepository;

@GetMapping("/client/{clientId}")
public ResponseEntity<?> getClientBookings(@PathVariable UUID clientId) {
    List<Booking> bookings =
bookingRepository.findByClientIdOrderByCreatedAtDesc(clientId);
    List<Map<String, Object>> response = new ArrayList<>();
    for (Booking booking : bookings) {
        Map<String, Object> map = new HashMap<>();
        map.put("id", booking.getId());
        // ... 10 more put() calls ...
        User technician =
userRepository.findById(booking.getTechnicianId()).orElse(null);
        if (technician != null) {
            map.put("technicianName", technician.getName());
            map.put("technicianContact", technician.getContactNo());
        }
        response.add(map);
    }
    return ResponseEntity.ok(response);
}
```

Facade Pattern:

```
// BookingController.java (AFTER) — delegates to facade
private final BookingService bookingService; // injected as
LoggingBookingService → BookingFacade

@GetMapping("/client/{clientId}")
public ResponseEntity<?> getClientBookings(@PathVariable UUID clientId) {
    try {
        return
ResponseEntity.ok(bookingService.getClientBookings(clientId));
    } catch (Exception e) {
        return ResponseEntity.badRequest().body("Failed to fetch bookings:
" + e.getMessage());
    }
}
```

Where it was applied

- BookingFacade now handles methods like createBooking(), getClientBookings(), getTechnicianBookings(), getBookingDetails(), and updateBookingStatus()
- BookingController now depends on BookingService, which is backed by the facade

Why this pattern fits
The Facade pattern is useful when several lower-level operations should be hidden behind one simple interface.
Instead of the controller dealing with repositories and enrichment itself, it calls the facade.

Improvement

- BookingController became much shorter and cleaner.
- Booking-related logic is now reusable in other parts of the system.
- Enrichment logic is centralized and easier to test.

REPORT 4 — Decorator Pattern

Original implementation
There was no logging in the booking flow.
If logging had been added directly inside the controller or facade, it would mix logging with business logic.

Problems

- There was no easy way to track booking actions in production.
- Adding logs directly into the main logic would make methods messy.
- Logging could not be removed easily without editing the core class.

Applied pattern:

```java
@Primary
@Service
public class LoggingBookingService implements BookingService {

    private final BookingService delegate;

    public LoggingBookingService(
            @Qualifier("bookingFacadeImpl") BookingService delegate) {
        this.delegate = delegate;
    }

    @Override
    public List<Map<String, Object>> getClientBookings(UUID clientId) {
        log("getClientBookings", "START clientId=" + clientId);
        List<Map<String, Object>> result =
delegate.getClientBookings(clientId);
        log("getClientBookings", "DONE  → " + result.size() + " bookings");
        return result;
    }

    private void log(String method, String message) {
        System.out.printf("[%s] [BookingService] %s — %s%n",
                LocalDateTime.now(), method, message);
    }
}
```

Decorator Pattern (Sample output on the console):

```
[2026-04-06T18:00:01] [BookingService] getClientBookings — START
clientId=abc-123
[2026-04-06T18:00:01] [BookingService] getClientBookings — DONE  → 3
bookings
```

Where it was applied

- BookingService as the shared interface
- LoggingBookingService as the decorator
- BookingFacade as the real implementation

Why this pattern fits

The Decorator pattern lets you add extra behavior, like logging, without changing the original business class.

Improvement

- Booking operations now automatically produce logs.
- BookingFacade stays clean because logging is kept separate.
- More decorators can be added later, such as metrics or monitoring.

REPORT 5 — Strategy Pattern

Original implementation
BookingController.updateBookingStatus() checked valid statuses using a hard-coded list directly in the controller.

Problems

- Validation logic was placed in the wrong layer.
- Adding more rules would require editing the controller again.
- Different validation rules could not be swapped easily.
- This made the code less flexible.

Applied pattern:

```java
// BookingController.java (BEFORE)
String newStatus = request.get("status");
if (newStatus == null || !Arrays.asList(
        "pending", "confirmed", "in_progress", "completed", "cancelled")
        .contains(newStatus)) {
    return ResponseEntity.badRequest().body("Invalid status");
}
```

Strategy Pattern:

```java
// BookingStatusValidator.java (interface)
public interface BookingStatusValidator {
    boolean isValidTransition(String currentStatus, String newStatus);
}

// DefaultStatusValidator.java (concrete strategy)
@Component
public class DefaultStatusValidator implements BookingStatusValidator {
    private static final List<String> VALID_STATUSES =

Arrays.asList("pending","confirmed","in_progress","completed","cancelled");

    @Override
    public boolean isValidTransition(String currentStatus, String
newStatus) {
        return newStatus != null && VALID_STATUSES.contains(newStatus);
    }
}

// BookingFacade.java — uses strategy
if (!statusValidator.isValidTransition(booking.getStatus(), newStatus)) {
    throw new IllegalArgumentException("Invalid status transition: " +
newStatus);
}
```

Where it was applied

- BookingStatusValidator as the interface
- DefaultStatusValidator as the current validation rule
- Used inside BookingFacade.updateBookingStatus()

Why this pattern fits
The Strategy pattern is good when you want to separate a rule or algorithm from the class using it.
This makes it easier to replace or extend the validation later.

Improvement

- Status validation is now separate from the controller and facade logic.
- New validation rules can be added later, such as rules based on user role.
- Validation can now be tested on its own.

REPORT 6 — Observer Pattern

Original implementation
BookingController.updateBookingStatus() only updated the status and saved the booking.
It did not trigger any other action after that.

Problems

- No notification happened when a booking status changed.
- Adding email or push notifications would require changing the same controller method.
- More side effects would make that method bigger and harder to maintain.
- One notification type could not be removed easily without touching core logic.

Applied pattern:

```java
// BookingFacade.java
private final ApplicationEventPublisher eventPublisher;

public void updateBookingStatus(UUID bookingId, String newStatus) {
    Booking booking = bookingRepository.findById(bookingId)
            .orElseThrow(() -> new RuntimeException("Booking not found"));

    if (!statusValidator.isValidTransition(booking.getStatus(), newStatus))
{
        throw new IllegalArgumentException("Invalid status transition: " +
newStatus);
    }

    String oldStatus = booking.getStatus();
    booking.setStatus(newStatus);
    bookingRepository.save(booking);

    // Observer Pattern: publish event — listeners react independently
    eventPublisher.publishEvent(
            new BookingStatusChangedEvent(this, bookingId, oldStatus,
newStatus)
    );
}
```

Observer Pattern:

```java
// BookingNotificationListener.java
@Component
public class BookingNotificationListener {

    @EventListener
    public void handleBookingStatusChanged(BookingStatusChangedEvent event)
{
        System.out.printf(
            "[%s] [NOTIFICATION] Booking %s status changed: %s → %s%n",
            LocalDateTime.now(),
            event.getBookingId(),
            event.getOldStatus(),
            event.getNewStatus()
        );
    }
}
```

Where it was applied

- BookingStatusChangedEvent as the event
- BookingNotificationListener as the observer/listener
- BookingFacade as the publisher of the event

Why this pattern fits
The Observer pattern allows one part of the system to announce an event, while other parts react to it separately.
This keeps notification logic independent from business logic.

Improvement

- Every booking status change now triggers a notification automatically.
- New notification types like email, SMS, or push can be added without changing the facade.
- The booking logic and notification logic stay separate.